OS Practical No. 7

Aim : Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Code :

```
  GNU nano 8.7                                                    bankers.c
#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {

    int allocation[P][R] = {
        {0, 0, 1, 4},   // P0
        {0, 6, 3, 2},   // P1
        {0, 0, 1, 2},   // P2
        {1, 0, 0, 0},   // P3
        {1, 3, 5, 4}    // P4
    };

    int max[P][R] = {
        {0, 6, 5, 6},   // P0
        {0, 6, 5, 2},   // P1
        {0, 0, 1, 2},   // P2
        {1, 7, 5, 0},   // P3
        {2, 3, 5, 6}    // P4
    };

    int available[R] = {1, 6, 2, 0};

    int need[P][R];
    int finish[P] = {0};
    int safeSequence[P];
    int work[R];

    // Calculate NEED matrix
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Copy available to work
    for(int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while(count < P) {
        int found = 0;

                                                  [ Read 85 lines ]
^G Help      ^O Write Out   ^F Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-]
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^B
```

```
GNU nano 8.7                                                    bankers.c

    // Copy available to work
    for(int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while(count < P) {
        int found = 0;

        for(int i = 0; i < P; i++) {

            if(finish[i] == 0) {

                int possible = 1;

                for(int j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if(possible) {

                    for(int j = 0; j < R; j++)
                        work[j] += allocation[i][j];

                    safeSequence[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE state.\nSafe Sequence: ");
    for(int i = 0; i < P; i++)
        printf("P%d ", safeSequence[i]);

    printf("\n");

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

OUTPUT:

```
Student@DESKTOP-JJ8CFNB MINGW64 ~
$ nano bankers.c

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ gcc bankesr.c -o bankers
cc1.exe: fatal error: bankesr.c: No such file or directory
compilation terminated.

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ gcc bankers.c -o bankers

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ ./bankers

System is in SAFE state.
Safe Sequence: P1 P2 P3 P4 P0

Student@DESKTOP-JJ8CFNB MINGW64 ~
$
```